

Introduction à React

Historique et contexte de développement

- Origine de React

- React a été développé par Facebook en interne en 2011 pour résoudre les problèmes de performances et de maintenabilité rencontrés dans le développement d'interfaces utilisateur complexes.

- Publication en open source

- Facebook a publié React en open source en 2013, ce qui a permis à la communauté de développeurs de contribuer au projet et de l'adopter rapidement.

- Adoption massive

- En raison de sa simplicité, de ses performances élevées et de sa réutilisabilité, React est rapidement devenu l'une des bibliothèques JavaScript les plus populaires pour le développement d'applications web.

Avantages de l'utilisation de React

- Virtual DOM

- React utilise un concept appelé Virtual DOM pour améliorer les performances des applications en minimisant les manipulations directes du DOM.

- Composants réutilisables

- React encourage la construction d'interfaces utilisateur modulaires en utilisant des composants réutilisables, ce qui facilite le développement, la maintenance et l'évolutivité des applications.

- JSX (JavaScript XML)

- JSX est une extension de syntaxe qui permet d'écrire du code HTML à l'intérieur de JavaScript, ce qui rend le code plus lisible et plus facile à maintenir.

- Unidirectional data flow

- React suit un modèle de flux de données unidirectionnel, ce qui facilite le suivi des données et la détection des erreurs dans l'application.

- Large écosystème

- React est accompagné d'un écosystème riche de bibliothèques tierces, d'outils de développement et de solutions prêtes à l'emploi qui facilitent le développement d'applications complexes.

Création d'un nouveau projet React avec Create React App

- Create React App

- Create React App est un outil officiel de Facebook qui permet de créer rapidement et facilement un nouveau projet React, préconfiguré avec les outils et les configurations nécessaires.

- Installation

- Pour créer un nouveau projet React, les étudiants doivent ouvrir leur terminal et exécuter la commande suivante :

```
npx create-react-app nom-du-projet
```

- Cela va créer un nouveau répertoire nommé "nom-du-projet" contenant une structure de projet React de base.
-

Structure de base d'une application React

- Structure des fichiers et des dossiers

- Le projet créé avec Create React App comprend une structure de fichiers et de dossiers bien organisée, avec les éléments suivants :

```

nom-du-projet/
├── node_modules/      # Répertoire des dépendances du projet (généré
automatiquement)
├── public/            # Contient les fichiers statiques accessibles
au public (index.html, favicon.ico, etc.)
├── src/               # Contient le code source de l'application
React
|   ├── components/    # Dossier pour les composants réutilisables de
l'application
|   ├── App.js         # Point d'entrée de l'application, composant
racine
|   ├── index.js       # Point d'entrée de l'application, fichier
principal
|   └── ...            # Autres fichiers et dossiers
└── package.json       # Fichier de configuration npm contenant les
métadonnées du projet et les dépendances
└── package-lock.json # Fichier de verrouillage des versions des
dépendances (généré automatiquement)

```

Création de composants fonctionnels et de classe

Différences entre les deux approches

- Composants fonctionnels

- Les composants fonctionnels sont des fonctions JavaScript qui renvoient du JSX.
- Ils sont simples, légers et faciles à lire.
- Ils ne possèdent pas de state ni de méthodes de cycle de vie.

Exemple:

```
import React from 'react';

// Définition d'un composant fonctionnel nommé FunctionalComponent
const FunctionalComponent = () => {
  // Le composant fonctionnel renvoie du JSX
  return (
    <div>
      <h1>Composant fonctionnel</h1>
      <p>Ce composant est fonctionnel et ne possède pas de state ni de
méthodes de cycle de vie.</p>
    </div>
  );
};

export default FunctionalComponent;
```

Dans cet exemple :

- Nous importons React depuis la bibliothèque React.
- Nous définissons un composant fonctionnel nommé `FunctionalComponent` en utilisant une syntaxe de fonction fléchée `() => {}`.
- À l'intérieur de la fonction, nous retournons du JSX qui représente l'interface utilisateur du composant. Dans cet exemple, nous avons un `<div>` contenant un titre `<h1>` et un paragraphe `<p>`.
- Enfin, nous exportons le composant fonctionnel en utilisant `export default`.

- Composants de classe

- Les composants de classe sont des classes JavaScript qui étendent la classe `React.Component`.
- Ils peuvent avoir un state et des méthodes de cycle de vie.
- Ils sont utilisés lorsque le composant a besoin de gérer son propre état interne ou d'utiliser des méthodes de cycle de vie.

Exemple:

```
import React, { Component } from 'react';

// Définition d'un composant de classe nommé ClassComponent
class ClassComponent extends Component {
  // Constructeur du composant de classe
  constructor(props) {
    super(props);
    // Initialisation du state du composant
    this.state = {
      count: 0
    };
  }

  // Méthode appelée lorsqu'un bouton est cliqué pour incrémenter le
  // compteur
  handleClick = () => {
    // Mise à jour du state en utilisant this.setState()
    this.setState({ count: this.state.count + 1 });
  };

  // Méthode de rendu du composant
  render() {
    // Le composant de classe renvoie du JSX
    return (
      <div>
        <h1>Composant de classe</h1>
        <p>Compteur : {this.state.count}</p>
        /* Bouton qui appelle la méthode handleClick lorsqu'il est
        cliqué */
        <button onClick={this.handleClick}>Incrémenter</button>
      </div>
    );
  }
}
```

```
    }  
  }  
  
  export default ClassComponent;
```

Dans cet exemple :

- Nous importons `React` et `Component` depuis la bibliothèque `React`.
- Nous définissons un composant de classe nommé `ClassComponent` en étendant la classe `Component`.
- Nous définissons un constructeur pour le composant de classe où nous initialisons le state du composant.
- Nous définissons une méthode `handleClick` qui sera appelée lorsque le bouton est cliqué. Cette méthode utilise `this.setState()` pour mettre à jour le state du composant.
- Nous définissons la méthode de rendu `render()` qui renvoie du JSX représentant l'interface utilisateur du composant. Dans cet exemple, nous affichons un titre, un compteur (`this.state.count`) et un bouton qui appelle la méthode `handleClick`.
- Enfin, nous exportons le composant de classe en utilisant `export default`.

Utilisation de state et de props

- State

- Le state est une propriété d'un composant qui permet de stocker et de gérer les données qui peuvent changer au fil du temps.
- Il est défini dans le constructeur du composant en utilisant `this.state` et peut être mis à jour avec `this.setState()`.
- Le state est privé au composant et ne peut pas être directement modifié en dehors du composant.

Depuis la version 16.18 de React, il est également possible d'utiliser le hook `useState()` qui permet aux composants fonctionnels de posséder un état local. Avant l'introduction des hooks, les composants fonctionnels étaient des composants "dumb" qui ne pouvaient pas gérer leur propre état interne. `useState()` permet de remédier à cela en ajoutant une fonctionnalité d'état aux composants fonctionnels sans avoir à les convertir en composants de classe.

Exemple:

```
import React, { useState } from 'react';

const Counter = () => {
  // Utilisation de useState pour créer un état local "count" avec une
  // valeur initiale de 0
  const [count, setCount] = useState(0);

  // Fonction appelée lorsque le bouton est cliqué pour incrémenter le
  // compteur
  const handleClick = () => {
    setCount(count + 1); // Mise à jour du state en incrémentant la
    // valeur actuelle de count
  };

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={handleClick}>Incrémenter</button>
    </div>
  );
};

export default Counter;
```

Dans cet exemple :

- Nous utilisons `useState(0)` pour créer un état local `count` avec une valeur initiale de 0.
- Lorsque le bouton est cliqué, la fonction `handleClick` est appelée, ce qui utilise `setCount` pour mettre à jour la valeur de `count` en incrémentant la valeur actuelle de `count`.
- La valeur actuelle de `count` est affichée dans le composant à l'aide de `{count}` dans le JSX.

useState rend la gestion de l'état dans les composants fonctionnels aussi simple et intuitive que dans les composants de classe, ce qui permet de créer des composants fonctionnels plus puissants et plus flexibles.

- Props

- Les props (propriétés) sont des données passées de parent à enfant via des attributs HTML-like.
- Ils permettent de rendre un composant configurable et réutilisable en lui fournissant des données externes.
- Les props sont en lecture seule et ne peuvent pas être modifiées par le composant enfant.

Exemple:

Message.jsx

```
import React from 'react';

// Composant fonctionnel qui affiche un message passé via les props
const Message = (props) => {
  return <p>{props.text}</p>;
};

export default Message;
```

App.js

```
import React from 'react';
import Message from './Message';

// Composant parent qui passe une prop au composant enfant
const App = () => {
  return <Message text="Bonjour React !" />;
};

export default App;
```

Dans cet exemple, le composant parent App passe une prop text au composant enfant Message. Le composant Message affiche ensuite le texte passé via les props.

En utilisant le state et les props, vous pouvez créer des composants React dynamiques, réutilisables et interagir avec eux de manière flexible en fonction des données fournies par les props et de l'état interne géré par le state.

Gestion des événements

Ajout de gestionnaires d'événements aux composants

Les gestionnaires d'événements sont des fonctions qui sont exécutées en réponse à des actions de l'utilisateur, telles que cliquer sur un bouton, saisir du texte dans un champ de formulaire, etc. En React, vous pouvez ajouter des gestionnaires d'événements aux éléments JSX en utilisant des attributs d'événement spécifiques, tels que `onClick`, `onChange`, `onSubmit`, etc.

Syntaxe

La syntaxe pour ajouter un gestionnaire d'événements à un élément JSX est la suivante :

```
<element onClick={handleClick}></element>
```

Dans cet exemple :

- `onClick` est un attribut d'événement qui définit la fonction à exécuter lorsque l'événement de clic se produit.
- `handleClick` est la fonction qui sera exécutée lorsque l'événement de clic se produit.

Exemple d'ajout de gestionnaire d'événements

Voici un exemple où nous ajoutons un gestionnaire d'événements de clic à un bouton :

```
import React from 'react';

const MyComponent = () => {
  // Fonction de gestion d'événements pour le clic du bouton
  const handleClick = () => {
    alert('Le bouton a été cliqué !');
  };
};
```

```
return (  
  <div>  
    <button onClick={handleClick}>Cliquez ici</button>  
  </div>  
)  
};  
  
export default MyComponent;
```

Dans cet exemple :

- Nous avons un composant fonctionnel MyComponent.
- Nous avons défini une fonction handleClick qui affiche une alerte lorsque le bouton est cliqué.
- Nous avons ajouté un gestionnaire d'événements de clic au bouton en utilisant l'attribut onClick, qui déclenchera l'exécution de la fonction handleClick lorsque le bouton est cliqué.

Cela permet d'ajouter des fonctionnalités interactives à vos composants React et de répondre aux actions de l'utilisateur de manière dynamique.

Communication entre composants

Passage de données via les props

Les props (propriétés) sont un mécanisme clé de React permettant de passer des données d'un composant parent à un composant enfant. Les composants React peuvent accepter des données externes sous forme de props, ce qui les rend configurables et réutilisables dans différentes parties de l'application.

Syntaxe

Pour passer des données d'un composant parent à un composant enfant via les props, vous devez définir des attributs dans l'élément JSX correspondant au composant enfant. Ces attributs seront accessibles dans le composant enfant sous forme de propriétés.

Exemple d'utilisation des props

Voici un exemple où nous passons des données d'un composant parent à un composant enfant via les props :

```
import React from 'react';

// Composant enfant qui affiche un message reçu via les props
const ChildComponent = (props) => {
  return <p>Message reçu : {props.message}</p>;
};

// Composant parent qui rend le composant enfant et lui passe des
données via les props
const ParentComponent = () => {
  return <ChildComponent message="Bonjour, je suis un message
passé via les props !" />;
};

export default ParentComponent;
```

Dans cet exemple :

- Nous avons un composant enfant `ChildComponent` qui reçoit un message via les props et l'affiche dans un paragraphe.
- Nous avons un composant parent `ParentComponent` qui rend le composant enfant `ChildComponent` et lui passe un message via les props en utilisant l'attribut `message`.
- Le message passé via les props est affiché dans le composant enfant lorsqu'il est rendu.

Cela démontre comment les données peuvent être transmises d'un composant parent à un composant enfant via les props, permettant ainsi une communication efficace entre les composants dans une application React.

Utilisation de useEffect

useEffect est un hook de React qui permet d'effectuer des opérations "side effects" (effets secondaires) dans des composants fonctionnels. Ces effets peuvent inclure des actions telles que la récupération de données à partir d'une API, la souscription à des événements, ou la mise à jour du DOM. useEffect est exécuté après chaque rendu du composant, y compris le premier rendu et les mises à jour subséquentes.

Importation : Pour utiliser useEffect, vous devez l'importer depuis la bibliothèque React :

```
import React, { useEffect } from 'react';
```

Utilisation:

```
useEffect(() => {  
  // Effectuer des opérations "side effects" ici  
}, [dependencies]);
```

- La fonction passée à useEffect est exécutée après chaque rendu.
- Vous pouvez optionnellement spécifier un tableau de dépendances. Si ce tableau est fourni, l'effet ne sera exécuté que si l'une des valeurs dans le tableau de dépendances change entre les rendus.

Exemple d'utilisation avec un appel API:

```
import React, { useState, useEffect } from 'react';

function MyComponent() {
  const [data, setData] = useState(null);

  useEffect(() => {
    // Fonction pour récupérer des données depuis une API
    const fetchData = async () => {
      const response = await fetch('https://api.example.com/data');
      const responseData = await response.json();
      setData(responseData);
    };

    // Appeler la fonction pour récupérer les données lors du montage du
    // composant
    fetchData();
  }, []); // Pas de dépendances, donc exécuté seulement une fois après
  // le montage initial

  return (
    <div>
      {/* Afficher les données récupérées */}
      {data && (
        <ul>
          {data.map(item => (
            <li key={item.id}>{item.name}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

```
export default MyComponent;
```

Dans cet exemple :

Nous utilisons `useState` pour gérer un état `data` qui stocke les données récupérées. Nous utilisons `useEffect` pour effectuer la requête API en utilisant `fetch` lorsque le composant est monté.

Le tableau de dépendances est vide (`[]`), donc l'effet est exécuté uniquement après le premier rendu du composant.

Exemple avec dépendances pour mettre à jour le titre de la page

Exemple avec dépendances pour modifier le titre de la page:

```
import React, { useState, useEffect } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  // Utilisation de useEffect avec une dépendance sur 'count'
  useEffect(() => {
    // Mettre à jour le titre de la page avec la valeur actuelle du
    // compteur
    document.title = `Compteur: ${count}`;
  }, [count]); // Dépendance: 'count'

  return (
    <div>
      <p>Compteur: {count}</p>
      <button onClick={() => setCount(count + 1)}>Incrémenter</button>
    </div>
  );
}

export default Counter;
```

Dans cet exemple :

Nous utilisons `useState` pour gérer l'état du compteur.

Nous utilisons `useEffect` pour mettre à jour le titre de la page chaque fois que la valeur du compteur change.

Nous passons `[count]` en tant que tableau de dépendances à `useEffect`, ce qui signifie que l'effet sera exécuté chaque fois que la valeur de `count` change.

Lorsque vous cliquez sur le bouton "Incrémenter", le compteur augmente et le titre de la page est mis à jour en conséquence. Cela montre comment utiliser `useEffect` avec des

dépendances pour réagir de manière dynamique aux changements d'état dans un composant React.